From security signals to reviewed, verified change

A customer workflow for OpenFix

Markus Weldon • Independent architecture proposal and prototype

01

UNDERSTAND

Scope the customer problem

02

PROVE

Validate evidence and behavior

03

FIX

Review the exact change

Start with the customer outcome

Let the provided scenario determine the scope. The billing example is only a practice vignette.

PROBLEM

Where is work stuck?

Detection noise, investigation, ownership, patch creation, or review?

SCOPE

One service, one issue class

Identify the security owner, service owner, approved data, and success criteria.

VALUE

Measure accepted fixes

Compare end-to-end effort, verified outcomes, review burden, and failure rate.

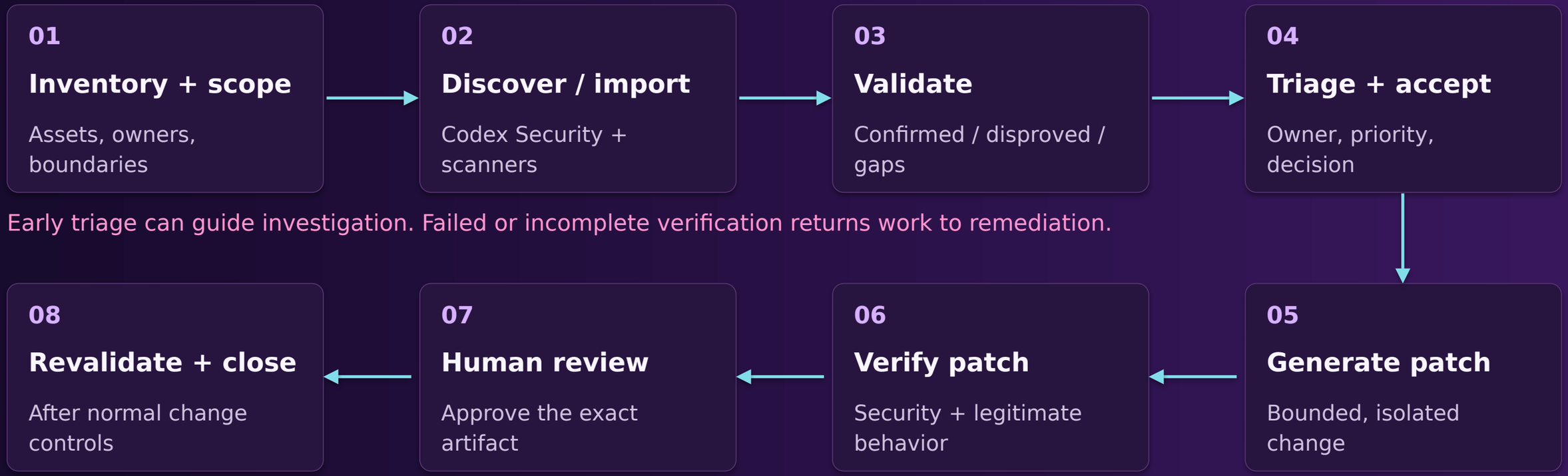
Choose the right capability

Start with native capabilities, then add the customer integration that is actually missing.

Defense Factory	Continuous operating model and reference architecture
Daybreak	Broader governed cyber-defense stack
Codex Security	Code discovery, finding evidence, validation and fixes
Promptfoo	AI application evaluation and adversarial behavior testing
OpenFix proposal	Customer context, evidence, ownership and change workflow

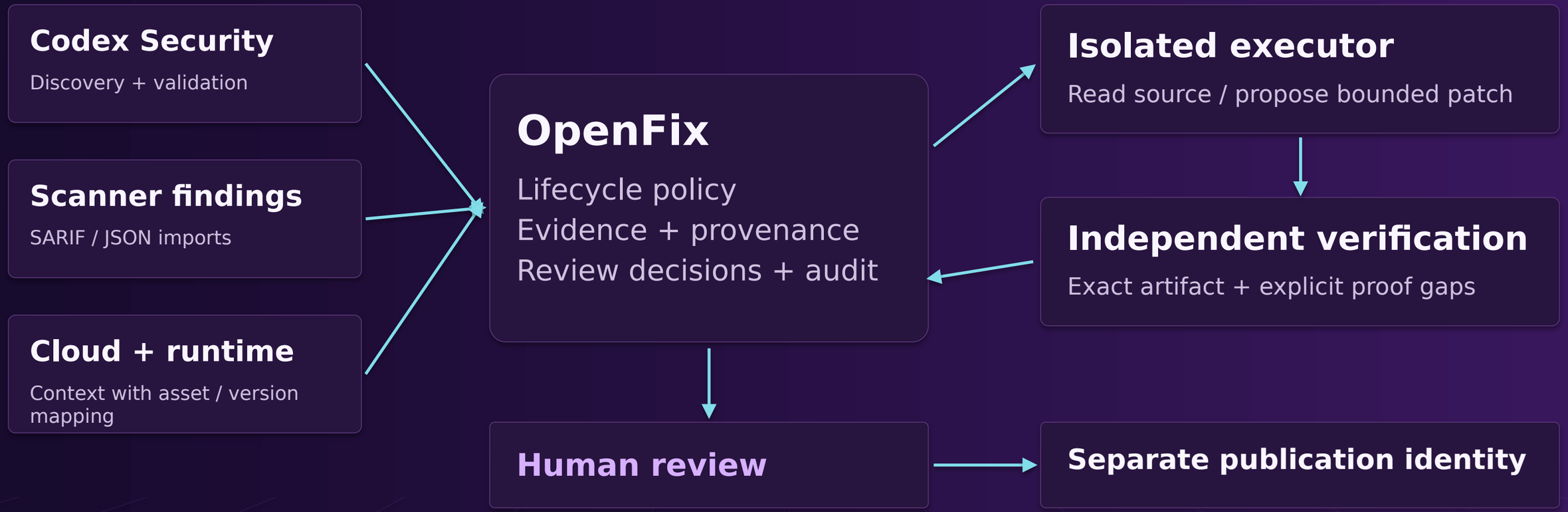
The complete security loop

Evidence, acceptance, verification, and closure are separate decisions.



A control plane with clear authority

Target architecture: integrate security capabilities without giving one process every permission.



Controls travel with the workflow

Assign authority to each action and bind approval to a specific artifact.

Read / investigate	Approved source and data; scoped execution
Propose a patch	Accepted finding; bounded isolated workspace
Verify	Independent result at the exact changed revision
Publish / merge	Separate credentials; reviewer and branch protection
Deploy / close	Normal change process; post-change evidence

A small pilot with a real decision

Agree the baseline, evidence, owners, and exit criteria before expanding.

BASELINE

Current workflow

Approved data, labeled examples, reviewer standards, investigator effort.

EVALUATE

Positive + negative cases

Verified acceptance, false fixes, proof gaps, latency, cost, review time.

DECIDE

Expand, revise, or stop

One service and issue class first. Define the next owner and decision date.

Lead an adaptive working session

No presentation is required. Use a visual only when it helps the customer conversation.

01	**Frame the problem**	Outcome, stakeholders, bottleneck, constraints
02	**Map the workflow**	Assets, tools, versions, evidence and handoffs
03	**Choose and bound**	Smallest useful AI step + identity / data boundaries
04	**Define a pilot**	Proof, baseline, owners, milestones and stop criteria
05	**Adapt and recommend**	Revise assumptions; explain the next decision

Leave with a concrete recommendation

A good customer conversation ends with an owned next step and visible uncertainty.

One service. One issue class. One accountable reviewer.

01

Agree the scope

Problem, assets, data, authority

02

Define the proof

Original issue + legitimate behavior

03

Choose the decision

Success measures + expansion gate

What exists, and what changes

The repository should make implementation status easy to inspect.

CURRENT V1

Remediation orchestrator

SARIF / JSON ingestion, policy routing, Cursor agents, PR CI observation.

TESTED PROTOTYPE

Lifecycle decision gates

Validation, triage, patch identity, verification, review, revalidation, closure.

PROPOSED INTEGRATION

Full customer workflow

Codex Security adapter, isolated jobs, durable evidence, workbench, publication.

The repo’s next three increments

Make one customer workflow operational before adding a broad product surface.

01 / INTEGRATE

Persistent security cases

Wire the case lifecycle into the actual CLI. Preserve evidence, revisions, owners and decisions.

02 / PROVE

One real capability path

Use a documented Codex Security path. Verify one bounded change and handle failure honestly.

03 / EVALUATE

Useful review and evidence

Add a workbench over real state. Evaluate agent behavior when needed; expand from measured proof.

Keep these decisions separate

A status label should express the evidence and authority actually present.

Candidate	A hypothesis worth investigating
Confirmed	Evidence supports the security finding
Accepted	A reviewer chooses owner, priority, and remediation scope
Verified patch	Agreed checks pass for the exact artifact
Closed	Change controls and post-change evidence support closure

From a signal to an attack path

Hypothetical issue: invoice lookup authenticates a caller but omits tenant ownership.

01

Attacker input

Invoice identifier

→

02

Control failure

Lookup omits tenant predicate

→

03

Supported impact

Another tenant invoice is returned

Context informs priority; the code path establishes the flaw.

Cloud exposure: internet-facing asset. Runtime: requests correlated to a known version.
Mapping gaps and unverified exploitation remain visible.

A useful workflow handles uncertainty

Show the branches as deliberately as the successful fix.

EVIDENCE SUPPORTS IT

Confirmed

Accept for a bounded remediation after reviewing scope and impact.

A CONTROL ALREADY HOLDS

Disproved

Record the evidence and dismissal reason. Do not spend a patch run.

PROOF IS INCOMPLETE

Needs context

Name the missing assumption, test, version mapping, or permission.

Separate access, usage, and deployment

Pro familiarization does not establish that every Security surface uses included allowance.

EXPLORE LOCALLY

ChatGPT sign-in

Learn the native finding, evidence and patch workflow. Confirm access and limits.

CLOUD SCANS

Separate billing

Token billing applies to new Cloud scans. Eligible scanning credits may apply; check settings.

CUSTOMER ROLLOUT

Operating requirements

Choose identities, data handling, isolation, budgets, verification and review controls.

Sources and practical boundaries

Public product documentation supports the workflow; the architecture and scenario are independent.

Defense Factory: process and architecture
https://openai.com/the-defense-factory/

Daybreak: broader capability stack
https://openai.com/daybreak/

Cloud setup and billing boundaries
https://learn.chatgpt.com/docs/security/faq

Finding and fix evidence
https://learn.chatgpt.com/docs/security/plugin/fix-findings

Promptfoo: adversarial agent evaluation
https://www.promptfoo.dev/docs/red-team/agents/